

Enterprise AI Platform — Developer Guide

June 2026

Audience: Developers and contributors who need to build, run, extend, test, and debug the platform. This is a practical guide — architectural rationale lives in the [Architecture Handbook](#), and design decisions are documented in the [ADR volume](#).

Contents

- 1. [Repository Structure](#)
- 2. [Development Environment](#)
- 3. [Building the Platform](#)
- 4. [Running the Platform](#)
- 5. [Configuration](#)
- 6. [Module Overview](#)
- 7. [Extending the Platform](#)
- 8. [Testing](#)
- 9. [Debugging](#)
- 10. [Observability](#)
- 11. [Performance](#)
- 12. [Contribution Guidelines](#)

1. Repository Structure

enterprise-ai-platform/	
— platform-audit/	Immutable audit log (leaf module)
— platform-auth/	JWT authentication + user management
— platform-document/	Document lifecycle + ingestion pipeline
— platform-search/	Hybrid search (keyword + vector + graph)
— platform-ai/	RAG orchestration, enrichment, evaluation, registries
— platform-neo4j/	Knowledge graph persistence (auto-generated)
— platform-workspace/	Workspace wizard + timeline
— platform-observability/	Micrometer metrics + health indicators
— platform-api/	REST controllers, Thymeleaf UI, assembly

— docs/	Architecture Handbook, Developer Guide, ADRs, diagrams
— test-corpus/	AI regression test documents
— e2e-tests/	Playwright browser tests
— docker-compose.yml	PostgreSQL + Qdrant + Neo4j (graph profile)
— pom.xml	Parent POM (Spring Boot 3.3.5, Java 21)

Dependency direction: `platform-api` depends on all platform modules. Platform modules depend on `platform-audit` (leaf). Nobody depends on `platform-api`. Compile-time enforcement via Maven.

Key configuration: `platform-api/src/main/resources/application.yml` — all `platform.*` properties. `platform-api/src/main/java/.../config/SearchInfrastructureConfig.java` — conditional bean wiring for providers.

2. Development Environment

Requirements:

- Java 21
- Maven 3.x
- Docker (for PostgreSQL, Qdrant, Neo4j)
- Node.js (for Playwright browser tests)

Quick start:

```
git clone <repo-url>
cd enterprise-ai-platform
docker compose up -d                # PostgreSQL + Qdrant
docker compose --profile graph up -d # + Neo4j (optional)
mvn spring-boot:run -pl platform-api # Start on :8080
```

IDE setup: Import as Maven project. The root `pom.xml` is the parent. All 9 modules are children. Use IntelliJ IDEA or Eclipse with Java 21 SDK.

3. Building the Platform

```
# Compile all modules
mvn compile

# Run all tests (unit + integration + architecture)
mvn verify

# Run tests excluding Playwright browser tests
mvn verify -DskipUITests
```

```
# Build without tests
mvn package -DskipTests

# Run a single test class
mvn test -pl platform-ai -Dtest=PromptRegistryTest

# Dependency analysis
mvn dependency:analyze
mvn dependency:tree -pl platform-search
```

Build artifacts: `platform-api/target/platform-api-0.1.0-SNAPSHOT.jar` is the runnable JAR.

4. Running the Platform

Docker services:

```
docker compose up -d          # PostgreSQL (5433) + Qdrant (6333/6334)
docker compose --profile graph up -d  # + Neo4j (7474/7687)
docker compose down           # Stop all services
```

Start the application:

```
mvn spring-boot:run -pl platform-api
```

Verify:

```
curl http://localhost:8080/actuator/health
curl http://localhost:8080/
```

The platform starts on port 8080. UI pages: `/dashboard`, `/documents/upload`, `/search`, `/workspaces`, `/ai`. Health endpoints: `/actuator/health`, `/actuator/prometheus`.

5. Configuration

All configuration lives in `application.yml` with `${ENV_VAR:default}` substitution:

```
platform:
  auth:
    jwt-secret: ${AUTH_JWT_SECRET:change-me}
  ai:
    ollama:
      base-url: ${OLLAMA_BASE_URL:http://localhost:11434}
      chat-model: ${OLLAMA_CHAT_MODEL:qwen2.5:14b}
```

```

neo4j:
  uri: ${NEO4J_URI:bolt://localhost:7687}
  username: ${NEO4J_USERNAME:neo4j}
  password: ${NEO4J_PASSWORD:password}
search:
  qdrant:
    host: ${QDRANT_HOST:localhost}
    collection: ${QDRANT_COLLECTION:enterprise_ai_chunks}

```

Key properties:

Property	Default	Purpose
<code>platform.ai.ollama.base-url</code>	<code>http://localhost:11434</code>	Ollama server
<code>platform.ai.openai.api-key</code>	(none)	Activates OpenAI provider when set
<code>platform.search.qdrant.host</code>	<code>localhost</code>	Qdrant server
<code>platform.neo4j.uri</code>	<code>bolt://localhost:7687</code>	Activates Neo4j graph
<code>platform.auth.jwt-secret</code>	(dev default)	JWT signing key — change in production

Conditional activation: Providers activate based on property presence. No Qdrant host → no vector search. No API key → no OpenAI provider. The platform starts with only PostgreSQL configured.

6. Module Overview

Each module has a single responsibility. Here is what you need to know to work in each one.

platform-audit (leaf module)

- `AuditService` — emit and query audit events
- `CorrelationIdFilter` — injects X-Correlation-Id into every HTTP response
- JDBC implementation via `JdbcAuditEventRepository`

platform-auth

- `AuthFacade` — register, login, refresh, logout
- `JwtTokenService` — HS256 signing with Nimbus JOSE
- `SecurityConfiguration` — stateless API + session form login

platform-document

- `DocumentFacade` — full document lifecycle CRUD
- `TextExtractionService` — PDF (PDFBox), DOCX (POI), HTML (JSoup)
- `DocumentIngestionWorker` — scheduled job processor (10s poll)

platform-search

- `SearchFacade` — public search API
- `HybridRetrievalService` — keyword + vector + graph fusion
- `KeywordSearchProvider` — JPA/TF scoring
- `VectorSearchProvider` -> `QdrantVectorSearchProvider` (or `NoOp`)
- `GraphSearchProvider` -> `Neo4jGraphSearchAdapter` (or `NoOp`)
- `SentenceAwareChunkingStrategy` — 1200-char target, 150-char overlap

platform-ai (the largest module)

- `AiFacade` / `AiService` — full 12-step inference pipeline
- `ChatCompletionProvider` -> `OllamaChatProvider`, `OpenAiChatProvider`
- `PromptRegistry` / `DefaultPromptRegistry` — 8 prompts, 10 categories
- `ModelCapabilityRegistry` / `DefaultModelCapabilityRegistry` — 6 models
- `ProviderRouter` / `DefaultProviderRouter` — 4-tier selection
- `EnrichmentService` / `DefaultEnrichmentService` — entity + concept extraction
- `EvaluationService` / `DefaultEvaluationService` — quality scoring
- `RetrievalOrchestrator` / `DefaultRetrievalOrchestrator` — intent-driven strategy
- `WorkflowEngine` / `DefaultWorkflowEngine` — 2 pre-registered workflows
- `DomainConfiguration` — SPI for domain-specific rules

platform-neo4j

- `GraphEnrichmentService` — persist, traverse, find related documents
- `NodeProvenance` — metadata on every node and relationship
- Activated by `platform.neo4j.uri` property

platform-workspace

- `WorkspaceService` — CRUD, phase transitions, document linking
- `WorkspaceOrchestrator` — phase handler registration
- 5-phase wizard: SETUP → INGESTION → ANALYSIS → REVIEW → COMPLETE

platform-observability

- `AiMetrics` — timers, counters, gauges for all AI operations
- `OllamaHealthIndicator`, `QdrantHealthIndicator` — health checks
- Prometheus endpoint at `/actuator/prometheus`

platform-api (assembly)

- REST controllers in `com.cognitera.platform.api.web`
- Thymeleaf controllers in `com.cognitera.platform.web`
- `SearchInfrastructureConfig` — conditional bean wiring for all providers

7. Extending the Platform

Adding an AI Provider

1. Implement `ChatCompletionProvider` (3 methods: `providerName`, `isAvailable`, `complete`)
2. Add `@Component` + `@ConditionalOnProperty`
3. Register model capabilities in `DefaultModelCapabilityRegistry`
4. The `ProviderRouter` discovers it automatically via `List<ChatCompletionProvider>` injection

```
@Component
@ConditionalOnProperty(name = "platform.ai.your-provider.api-key")
public class YourChatProvider implements ChatCompletionProvider {
    @Override public String providerName() { return "your-provider"; }
    @Override public boolean isAvailable() { return true; }
    @Override public String complete(String prompt, ModelCapabilities caps) {
        // HTTP call to your provider's chat API
    }
}
```

Adding a Prompt

```
promptRegistry.register(new PromptTemplate("my-prompt", 1,
    PromptTemplate.Category.RETRIEVAL, "Description",
    "Template with {{variable}}", List.of("variable"),
    "text", List.of("*"), 0.3, List.of(), Map.of()));
```

Adding a Workflow

```
workflowEngine.start("my-workflow", Map.of("key", "value"));
```

Workflows are defined in `DefaultWorkflowEngine` 's constructor. New workflows can be added by calling `register()` in any `@PostConstruct` or configuration bean.

Adding a Domain

Implement `DomainConfiguration`. Provide as `@Component` or `@Bean`. The AI pipeline picks it up automatically when injected.

Adding a Retrieval Source

Implement `GraphSearchProvider` (or a new SPI if the existing one doesn't fit). Add `@Component` + conditional activation. `DefaultHybridRetrievalService` accepts `GraphSearchProvider` via constructor injection.

8. Testing

Test layers:

```
mvn test                # Unit tests (44)
mvn verify              # Unit + Integration + Architecture (157)
mvn verify -Pui-tests   # + Playwright browser tests
```

Test locations:

Layer	Location	Spring Context?
Unit	<code>platform-ai/src/test/.../unit/</code>	No
Integration	<code>platform-api/src/test/.../ai/</code>	Yes
Architecture	<code>platform-api/src/test/.../architecture/</code>	Yes
Resilience	<code>platform-api/src/test/.../resilience/</code>	No
Contract	<code>platform-ai/src/test/.../contract/</code>	No
Playwright	<code>e2e-tests/playwright/</code>	Requires running app

Writing tests:

- Unit tests: No Spring context. Mock dependencies. Test behavior.
- Integration tests: `@SpringBootTest` with H2 in-memory database.
- Architecture tests: Validate end-to-end flows with real Spring context.
- Contract tests: Extend `ChatCompletionProviderContract` for new providers.
- Playwright tests: Run `npm playwright test` with app running on :8080.

Test corpus: `test-corpus/` contains representative documents (technical spec, financial report, contract). `platform-api/src/test/resources/test-corpus/` is the classpath copy used in tests.

9. Debugging

Enable debug logging:

```
logging:
  level:
    com.cognitera.platform: DEBUG
```

Trace AI pipeline execution:

The `RetrievalOrchestrationResult.traceLog()` contains step-by-step decisions from the retrieval orchestrator. Inspect it in `DefaultRetrievalOrchestrator.orchestrate()`.

Provider routing:

The `DefaultProviderRouter` logs every routing decision at DEBUG level: which tier matched, which provider was selected, and why each tier was skipped.

Health checks:

`/actuator/health` shows the status of Ollama, Qdrant, and AI providers. Use this to verify infrastructure connectivity.

Common issues:

- *No providers available:* Check `platform.ai.ollama.base-url` or `OPENAI_API_KEY`
 - *No vector search results:* Check `platform.search.qdrant.host` — Qdrant may be down
 - *Graph operations failing:* Check `platform.neo4j.uri` — Neo4j may not be started
 - *Enrichment not running:* Works without LLM via regex fallback — check logs for extraction errors
-

10. Observability

Metrics endpoints:

- `/actuator/health` — Health status (with details)
- `/actuator/prometheus` — Prometheus metrics
- `/actuator/metrics` — All available metrics
- `/actuator/metrics/ai.inference.duration` — Specific metric

Key AI metrics:

Metric	Type	Description
<code>ai.inference.duration</code>	Timer	LLM inference latency by provider/model
<code>ai.embedding.duration</code>	Timer	Embedding generation latency
<code>ai.retrieval.duration</code>	Timer	Retrieval latency by mode
<code>ai.enrichment.duration</code>	Timer	Semantic enrichment latency
<code>ai.provider.available</code>	Gauge	Provider health (0=down, 1=up)

Logging: Structured logging via SLF4J with MDC correlation IDs (`X-Correlation-Id` header). All AI operations log at INFO level for audit trails, DEBUG level for diagnostics.

11. Performance

This is a reference implementation. It has not been benchmarked. Here are the architectural characteristics:

- **Chunking:** $O(n)$ linear scan, sentence-boundary-aware
- **Embedding:** Sequential — parallelizable via Java 21 virtual threads
- **Retrieval fusion:** $O(k+v+g)$ where k,v,g are candidate counts
- **Graph traversal:** 2-hop max, configurable via `GraphEnrichmentService.traverse()`
- **LLM timeout:** Configurable via `platform.ai.ollama.request-timeout` (default 120s)
- **Ingestion:** 10 documents per polling cycle, configurable via `platform.document.ingestion.poll-delay-ms`

No caching layer exists. Every retrieval re-executes. This is appropriate for a reference implementation — production deployments should add a caching layer.

12. Contribution Guidelines

Code style:

- Java 21, UTF-8 encoding
- Constructor injection — no `@Autowired` on fields
- Immutable records where possible
- Interfaces for public APIs, package-private for internals
- `@ConditionalOnProperty` for optional beans
- `ObjectProvider<T>` for optional dependencies

Commit messages:

- Present tense, imperative mood ("Add X" not "Added X")
- Architectural changes reference the relevant ADR

Pull requests:

- All tests must pass: `mvn verify`
- New subsystems need architecture tests
- New providers need contract tests
- Documentation updates if public APIs change

Versioning: Semantic versioning. Current: `1.0.0`. The main branch is `master`. Feature branches: `feature/description`. Release tags: `v1.0.0`, `v1.0.1`, etc.

Review checklist:

- ☐ Tests pass (`mvn verify`)
- ☐ New public APIs documented
- ☐ Conditional activation for optional features
- ☐ Graceful degradation when dependencies unavailable
- ☐ Health indicators updated
- ☐ No unused dependencies (`mvn dependency:analyze`)
- ☐ ADR created for architectural decisions

Further reading: The [Architecture Handbook](#) explains the design philosophy and trade-offs. The [ADR volume](#) documents every major decision.